Experiment9

1) Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <sys/stat.h>

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

/* File copy using low-level system calls */
long copy_syscall(const char *src, const char *dst, size_t bufsize)
{
    int in = open(src, O_RDONLY);
    if (in < 0) { perror("open src"); return -1; }
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { perror("open dst"); close(in); return -1; }

    off_t size = lseek(in, 0, SEEK_END);   /* find file size   */
    lseek(in, 0, SEEK_SET);                /* rewind to start  */
    (void)size;

    char *buf = malloc(bufsize);
    ssize_t n;
    long total = 0;
    while ((n = read(in, buf, bufsize)) > 0) {
        if (write(out, buf, n) != n) { perror("write"); break; }
        total += n;
    }
    free(buf);
    close(in);
    close(out);
    return total;
}

/* File copy using standard library functions */
long copy_stdio(const char *src, const char *dst, size_t bufsize)
{
    FILE *in = fopen(src, "rb");
    if (!in) { perror("fopen src"); return -1; }
    FILE *out = fopen(dst, "wb");
    if (!out) { perror("fopen dst"); fclose(in); return -1; }

    char *buf = malloc(bufsize);
    size_t n;
    long total = 0;
    while ((n = fread(buf, 1, bufsize, in)) > 0) {
        if (fwrite(buf, 1, n, out) != n) { perror("fwrite"); break; }
        total += n;
    }
    free(buf);
    fclose(in);
    fclose(out);
    return total;
```

```
}

int main(int argc, char *argv[])
{
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }
    size_t bufsize = 4096;

    double t1 = now();
    long b1 = copy_syscall(argv[1], argv[2], bufsize);
    double t2 = now();
    long b2 = copy_stdio(argv[1], argv[2], bufsize);
    double t3 = now();

    if (b1 < 0 || b2 < 0 || b1 != b2) {
        printf("Copy error\n");
        return 1;
    }
    printf("Bytes copied               : %ld\n", b1);
    printf("open/read/write time       : %.4f s\n", t2 - t1);
    printf("fopen/fread/fwrite time    : %.4f s\n", t3 - t2);
    return 0;
}
```

OUTPUT :

```
charith@DESKTOP-RBQQQSS:~$ nano exp9.1.c
charith@DESKTOP-RBQQQSS:~$ gcc exp9.1.c -o exp9.1
charith@DESKTOP-RBQQQSS:~$ head -c 64M /dev/urandom > source.dat
charith@DESKTOP-RBQQQSS:~$ ./exp9.1 source.dat copy.dat
Bytes copied               : 67108864
open/read/write time       : 0.0412 s
fopen/fread/fwrite time    : 0.0923 s
charith@DESKTOP-RBQQQSS:~$ cmp source.dat copy.dat && echo "Files are identical"
Files are identical
charith@DESKTOP-RBQQQSS:~$
```

2) Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(void)
{
    /* Create a small input file */
    FILE *f = fopen("input.txt", "w");
    fprintf(f, "Linux\nOperating Systems\nSystems Programming\nDup2\n");
    fclose(f);

    /* PART 1: redirect standard output to a file */
    printf("PART 1: Redirecting stdout to output.txt using dup2()\n");
    fflush(stdout);

    int saved_stdout = dup(STDOUT_FILENO);          /* save terminal */
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); exit(1); }
```

```
    dup2(fd, STDOUT_FILENO);                        /* fd 1 -> file  */
    close(fd);

    printf("This line goes to output.txt, not the screen.\n");
    printf("File descriptor 1 now points to the file.\n");
    fflush(stdout);

    dup2(saved_stdout, STDOUT_FILENO);              /* restore stdout */
    close(saved_stdout);
    printf("stdout restored to the terminal.\n\n");

    /* PART 2: redirect standard input from a file */
    printf("PART 2: Redirecting stdin from input.txt using dup2()\n");
    int saved_stdin = dup(STDIN_FILENO);
    int in = open("input.txt", O_RDONLY);
    if (in < 0) { perror("open"); exit(1); }
    dup2(in, STDIN_FILENO);                         /* fd 0 -> file  */
    close(in);

    char line[100];
    while (fgets(line, sizeof(line), stdin) != NULL)
        printf("  read from stdin : %s", line);

    clearerr(stdin);
    dup2(saved_stdin, STDIN_FILENO);                /* restore stdin */
    close(saved_stdin);
    printf("stdin restored to the keyboard.\n\n");

    /* PART 3: how a shell runs  sort < input.txt > sorted.txt */
    printf("PART 3: Shell style redirection  ->  sort < input.txt > sorted.txt\n");
    fflush(stdout);

    pid_t pid = fork();
    if (pid == 0) {                                 /* child process */
        int infd  = open("input.txt", O_RDONLY);
        int outfd = open("sorted.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        dup2(infd, STDIN_FILENO);
        dup2(outfd, STDOUT_FILENO);
        close(infd);
        close(outfd);
        execlp("sort", "sort", (char *)NULL);       /* replace child with sort */
        perror("execlp");
        _exit(1);
    }
    int status;
    waitpid(pid, &status, 0);
    printf("Child (PID %d) finished with exit status %d.\n", pid, WEXITSTATUS(status));
    return 0;
}
```

OUTPUT :

```
charith@DESKTOP-RBQQQSS:~$ nano exp9.2.c
charith@DESKTOP-RBQQQSS:~$ gcc exp9.2.c -o exp9.2
charith@DESKTOP-RBQQQSS:~$ ./exp9.2
PART 1: Redirecting stdout to output.txt using dup2()
stdout restored to the terminal.

PART 2: Redirecting stdin from input.txt using dup2()
  read from stdin : Linux
  read from stdin : Operating Systems
  read from stdin : Systems Programming
  read from stdin : Dup2
stdin restored to the keyboard.

PART 3: Shell style redirection  ->  sort < input.txt > sorted.txt
Child (PID 264) finished with exit status 0.
charith@DESKTOP-RBQQQSS:~$ cat output.txt
This line goes to output.txt, not the screen.
File descriptor 1 now points to the file.
charith@DESKTOP-RBQQQSS:~$ cat sorted.txt
Dup2
Linux
Operating Systems
Systems Programming
charith@DESKTOP-RBQQQSS:~$
```